```
import numpy as np
from queue import PriorityQueue

def a_star_search(graph, start, goal, h):
    open_set = PriorityQueue()
    open_set.put((0, start))
    g_score = {node: float('inf') for node in graph}
    g_score[start] = 0
    came_from = {}

    while not open_set.empty():
        current_f, current = open_set.get()

        if current == goal:
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
            path.reverse()
            return path, g_score[goal]

        for neighbor, cost in graph[current]:
            new_cost = g_score[current] + cost
            if new_cost < g_score[neighbor]:
                g_score[neighbor] = new_cost
                priority = new_cost + h(neighbor)
                open_set.put((priority, neighbor))
                came_from[neighbor] = current

    return None, float('inf')

# Define the graph and heuristic outside the function
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('A', 1), ('C', 2), ('D', 5)],
    'C': [('A', 4), ('B', 2), ('D', 1)],
    'D': [('B', 5), ('C', 1)],
}

heuristic = {
    'A': 7,
    'B': 6,
    'C': 2,
    'D': 0,
}

def h(node):
    return heuristic[node]

# Call the a_star_search function
path, cost = a_star_search(graph, 'A', 'D', h)
print("Path:", path)
```

```
print("cost:", cost)
```

```
Path: ['A', 'C', 'D']
cost: 5
```